Matriz de Auditoría de Calidad ISO/IEC 25010

Característica ISO 25010	Evidencia técnica en el código	Estado (Cumple / A corregir)
1. Adecuación funcional	**¿El backend en PHP procesa y persiste datos en MySQL vía PDO?** Sí. `api/src/db.php` abre la conexión PDO a MySQL (utf8mb4) y todas las consultas pasan por la función `consulta()`. Los controladores de `api/src/controladores/` registran usuarios, inician sesión, hacen el CRUD de disfraces y guardan los votos en las tablas `usuarios`, `disfraces` y `votos`. Las fotos se guardan en la columna `foto_blob`.	Cumple
2. Eficiencia de desempeño	**¿La API REST responde solicitudes JSON sin latencias desmedidas?** Sí. `GET /disfraces` devuelve la lista sin las fotos, así que pesa unos 1,5 KB. Las fotos se piden aparte y, si no cambiaron, el servidor responde 304 sin reenviarlas. El navegador achica cada foto a menos de 60 KB antes de subirla. La base tiene índices en `votos` y `disfraces`. `pruebas/probar_api.php` mide 20 pedidos seguidos y exige menos de 500 ms.	Cumple
3. Compatibilidad	**¿La arquitectura está desacoplada (Frontend JS Vanilla / Backend PHP)?** Sí. El frontend son páginas HTML con módulos de JavaScript sin frameworks (carpeta `js/`), que solo se comunican con el backend por `fetch()` y JSON (`js/api.js`). El backend no genera HTML: `api/index.php` es un router REST que responde JSON. La misma API la usan el navegador, `curl` y el script de pruebas.	Cumple
4. Fiabilidad	**¿Se capturan excepciones PDO y se devuelven códigos HTTP semánticos (200, 400, 500)?** Sí. Un manejador global en `api/src/bootstrap.php` convierte cada error en JSON con su código HTTP. Una `PDOException` por dato duplicado (SQLSTATE 23000) responde 409 y la falta de conexión a MySQL responde 503. Los votos se guardan en una transacción con `rollBack()`. Se usan los códigos 200, 201, 204, 400, 401, 403, 404, 405, 409, 415, 422, 500 y 503.	Cumple

5. Seguridad (OWASP)	**¿Las contraseñas usan BCRYPT y los Prepared Statements previenen SQLi?** Sí. Las contraseñas se guardan con `password_hash()` (bcrypt, hashes `2y 10$…`) y se verifican con `password_verify()`. Todas las consultas son sentencias preparadas de PDO con `ATTR_EMULATE_PREPARES = false`, y la prueba de inyección `admin' --` en el login es rechazada con 401. Además: tokens JWT firmados, control de rol en el servidor (403), `textContent` contra XSS y `.htaccess` que bloquea la configuración y el script SQL.	Cumple
6. Usabilidad / Accesibilidad	**¿Lighthouse marca 100/100 y la interfaz es 100 % operable por teclado?** Sí. Se corrigieron 14 puntos de WCAG 2.1 AA: contraste, nombres accesibles, foco perdido al votar y al editar, y títulos. Todo se maneja con Tab, Enter y Escape, con foco visible (`:focus-visible`) y un enlace "Ir al contenido". Las respuestas de `fetch()` se anuncian en contenedores con `aria-live="polite"` y el texto se inserta con `.textContent`. Lighthouse en Accesibilidad: 100/100 (ver captura).	Cumple
7. Mantenibilidad	**¿El código está estructurado en capas limpias y versionado en Git?** Sí. Está separado en capas: presentación (HTML y CSS), cliente (`js/api.js`, `js/sesion.js`, `js/ui.js` y un módulo por página), API (router `api/index.php`, controladores y servicios en archivos separados) y datos (`database/halloween.sql`). La configuración está centralizada en `config.php` y hay pruebas automáticas. El proyecto está versionado en Git, con un `.gitignore` que deja fuera las contraseñas locales.	Cumple